New York Times Comments Similarity using MinHashing and Locality Sensitive Hashing

Aliza Aziz
alizaaziz.ansari@studenti.unimi.it
82838A
Algorithms for Massive Datasets
Master in Computer Science
Università degli Studi di Milano, 2025–26
Instructor: Prof. Dario Malchiodi

Dataset

New York Times Articles & Comments 2020
Source: Kaggle — `benjaminawd/new-york-times-articles-comments-2020`
Access date: March 2025
Corpus used: NYT reader comments (`nyt-comments-part*.csv`)

Contents

1 Introduction **2**

2 Dataset **2**
2.1 Source and Access . . . 2
2.2 Corpus Selection . . . 2
2.3 Sampling Strategy . . . 3

3 Data Organisation **3**
3.1 Loading (Cell 3) . . . 3
3.2 Internal Data Structures (Cell 5) . . . 3

4 Pre-processing **3**
4.1 DataFrame Cleaning (Cell 4) . . . 3
4.2 Text Normalisation (Cell 7 — applied at shingling time) . . . 4

5 Algorithms and Implementation **4**
5.1 From-Scratch Hash Function (Cell 6) . . . 4
5.2 Step 1 — Shingling (Cell 7, MMDS §3.2) . . . 4
5.3 Step 2 — MinHashing (Cell 8, MMDS §3.3) . . . 5
5.3.1 Motivation . . . 5
5.3.2 Linear Congruential Generator . . . 5
5.3.3 MinHasher Class . . . 5
5.4 Step 3 — Locality Sensitive Hashing (Cell 9, MMDS §3.4) . . . 6
5.5 Step 4 — Candidate Verification (Cell 10) . . . 6

6 Scalability **7**
6.1 Theoretical Complexity . . . 7
6.2 Design Decisions for Scale . . . 7
6.3 Empirical Runtime . . . 7

7 Experiments **7**
7.1 Configuration . . . 8
7.2 Candidate Reduction . . . 8
7.3 Similar Pairs Found and Displayed (Cell 11) . . . 8
7.4 Similarity Distribution (Cell 12) . . . 8
7.5 Results Summary (Cell 13) . . . 9

8 Discussion **9**
8.1 Correctness . . . 9
8.2 Limitations . . . 9
8.3 Possible Extensions . . . 9

9 Conclusion **10**

A Global Parameters Reference **10**

B Declaration **10**

1. Introduction

Finding near-duplicate or highly similar documents within a large corpus is a foundational problem in data mining. Naive pairwise comparison requires $O(n^2)$ time: for a corpus of 30,000 comments there are already 449,985,000 possible pairs, making exhaustive comparison infeasible in practice.

This project implements the full *Finding Similar Items* pipeline from Chapter 3 of *Mining of Massive Datasets* (MMDS), applied to the New York Times Reader Comments 2020 dataset:

1. **Shingling** — convert each comment into a set of hashed 5-character substrings;
2. **MinHashing** — compress each shingle set into a 128-integer signature preserving Jaccard similarity;
3. **Locality Sensitive Hashing (LSH)** — identify candidate similar pairs via a banding scheme, avoiding all-pairs comparison.

All three algorithms are implemented **from scratch** in pure Python. The notebook is structured as 14 code cells with no markdown section headers. The only external libraries used are `kaggle` (dataset download), `pandas` (CSV loading), `numpy` (array storage), and `matplotlib` (plotting). The full set of imports is: `os`, `re`, `time`, `random`, `zipfile`, `glob`, `pandas`, `numpy`, `matplotlib.pyplot`, and `datetime`.

2. Dataset

2.1. Source and Access

Dataset Details

Name: New York Times Articles & Comments 2020
Source: Kaggle — `benjaminawd/new-york-times-articles-comments-2020`
Total size: approximately 2 GB (compressed zip)
Access date: March 2025
License: Public dataset for research and educational use

The dataset contains two types of files:

- `nyt-articles-2020.csv` — 33,574 news articles with fields including `uniqueID`, `headline`, `abstract`, `newsdesk`, `section`, `pub_date`, `word_count`, and `n_comments`.
- `nyt-comments-part0.csv` through `nyt-comments-part9.csv` — approximately 19.9 million reader comments across 10 part files, with fields including `commentID`, `commentBody`, `articleID`, `userDisplayName`, `createDate`, `recommendations`, `depth`, and `commentType`.

2.2. Corpus Selection

The project uses exclusively the **reader comments** (`commentBody`). Comments were chosen because they are far more numerous than articles (19.9 million vs. 33 thousand), naturally contain near-duplicates (copy-pasted replies, repeated boilerplate, the same comment posted under multiple articles), and their typical length of 80–500 characters is well-suited to character-level $k = 5$ shingling.

2.3. Sampling Strategy

Loading all 19.9 million comments at once is not feasible on a standard Colab instance. The notebook controls this with two global variables in Cell 1: `SUBSAMPLE = True` activates subsampling, and `MAX_COMMENTS = 30,000` sets the total comment budget. The 10 part files are read evenly — at most `per_file = MAX_COMMENTS // 10` rows per file via `nrows=per_file` (Cell 3). After cleaning, the corpus is randomly shuffled using `RANDOM_SEED = 42` and the first `MAX_COMMENTS` entries are kept, ensuring reproducibility. Setting `SUBSAMPLE = False` processes the full loaded data without truncation.

3. Data Organisation

3.1. Loading (Cell 3)

Comments are loaded with `pandas.read_csv` using:

- `usecols=['commentID', 'commentBody', 'articleID', 'userDisplayName']` — only the four columns actually needed are read;
- `nrows=per_file` — caps rows per file;
- `on_bad_lines='skip'` — tolerates malformed rows without crashing;
- `dtype=str` — prevents mis-casting of numeric-looking IDs.

The ten resulting chunks are concatenated into a single `DataFrame` with `pd.concat(chunks, ignore_index=True)`.

3.2. Internal Data Structures (Cell 5)

After cleaning and subsampling, the corpus is stored as parallel Python lists for $O(1)$ indexed access:

Variable	Type	Content
`docs`	`list[str]`	Lowercased comment text (algorithm input)
`doc_ids`	`list[str]`	Original `commentID`
`bodies`	`list[str]`	Original comment text (display only)
`article_refs`	`list[str]`	`articleID` each comment belongs to
`N_DOCS`	`int`	Total comments in the corpus
`shingle_sets`	`list[set[int]]`	Hashed k-shingle sets (Cell 7 output)
`signatures`	`list[list[int]]`	MinHash signatures (Cell 8 output)

Note: Cell 4 also computes `user_display` and Cell 5 includes it in `corpus_df`, but it is not extracted into a parallel list and is never used in any output — the corresponding print lines in `display_pair` (Cell 11) are commented out.

4. Pre-processing

Pre-processing occurs in two distinct stages.

4.1. DataFrame Cleaning (Cell 4)

Applied directly to `comments_df`:

1. **Drop nulls:** rows where `commentBody` is `NaN` are removed with `dropna`.
2. **Deduplicate:** exact duplicate `commentID` values are removed with `drop_duplicates`.
3. **Lowercase and strip:** a `text` column is created by applying `.astype(str).str.lower().str.strip()` to `commentBody`.
4. **Minimum length filter:** comments whose `text` has fewer than 80 characters are dropped. Short comments produce shingle sets too small for reliable Jaccard estimates.
5. **Display copy:** the original `commentBody` is preserved as `body_display`; `article_ref` is derived from `articleID` with `fillna('unknown')`.

4.2. Text Normalisation (Cell 7 — applied at shingling time)

The `preprocess_text` function is called inside `shingle_document`, not during DataFrame cleaning. It applies:

1. Convert to lowercase;
2. Replace all non-alphanumeric characters with a space via `re.sub`;
3. Collapse consecutive whitespace to a single space and strip leading/trailing whitespace.

This removes punctuation and HTML-encoded characters common in NYT comments (such as `&` and `"`) before shingles are extracted.

5. Algorithms and Implementation

5.1. From-Scratch Hash Function (Cell 6)

To avoid any external hashing library, a polynomial rolling hash is implemented as `hash_func`. For each character c in the input string, the accumulator is updated as $h \leftarrow (h \times 31 + \text{ord}(c)) \bmod p$, where $p = 2^{31} - 1 = 2{,}147{,}483{,}647$ (Mersenne prime). Two assertions verify correctness at runtime: `hash_func("hello") != hash_func("world")` and `hash_func("hello") == hash_func("hello")`.

```python
HASH_PRIME = 2_147_483_647    # Mersenne prime 2^31 - 1
HASH_BASE  = 31

def hash_func(text: str) -> int:
    h = 0
    for ch in text:
        h = (h * HASH_BASE + ord(ch)) % HASH_PRIME
    return h
```

Listing 1: Polynomial rolling hash (Cell 6)

5.2. Step 1 — Shingling (Cell 7, MMDS §3.2)

`shingle_document` calls `preprocess_text`, then slides a window of width $k = 5$ across the normalised text, adding `hash_func(window)` to a Python `set`. Using a set means each unique shingle is counted once regardless of repetitions.

`jaccard_similarity` computes the exact Jaccard coefficient $|A \cap B| / |A \cup B|$ for two shingle sets, returning 1.0 for two empty sets. This function is used in Cell 11 to compute true Jaccard for each displayed pair as a verification against the MinHash estimate.

After shingling all `N_DOCS` comments, the notebook prints the minimum, maximum, and mean shingle set size. The choice of $k = 5$ is motivated as follows:

- $k < 4$: windows such as `"the"` and `"ing"` are so common in English that unrelated comments share many, causing false matches;
- $k = 5$: discriminative enough to separate unrelated comments, short enough that similar 80–300-character comments share multiple shingles;
- $k > 7$: shingle sets become sparse for short comments, causing true matches to be missed.

5.3. Step 2 — MinHashing (Cell 8, MMDS §3.3)

5.3.1. Motivation

Direct Jaccard comparison of all comment pairs requires $O(n^2 \cdot |S|)$ work. MinHash compresses each shingle set into a fixed-length signature of `N_HASH = 128` integers while preserving Jaccard similarity in expectation: the fraction of signature positions where two signatures agree is an unbiased estimator of their Jaccard similarity.

5.3.2. Linear Congruential Generator

Random hash coefficients are generated entirely by the `LCG` class, using no call to `numpy.random` or Python's `random` module. The recurrence is $x_{n+1} = (A \cdot x_n + C) \bmod M$ with constants $A = 1{,}664{,}525$, $C = 1{,}013{,}904{,}223$, $M = 2^{32}$ from *Numerical Recipes*.

```python
class LCG:
    A = 1_664_525;  C = 1_013_904_223;  M = 2 ** 32

    def __init__(self, seed: int):
        self.state = seed % self.M

    def next_int(self) -> int:
        self.state = (self.A * self.state + self.C) % self.M
        return self.state
```

Listing 2: From-scratch LCG (Cell 8)

5.3.3. MinHasher Class

`MinHasher` uses the LCG to generate `N_HASH = 128` pairs (a_i, b_i). For each hash function i, `signature()` computes the minimum of $(a_i \cdot x + b_i) \bmod p$ over all shingles x in the document, returning a list of 128 integers. `estimated_jaccard` counts the fraction of the 128 positions where two signatures agree. With $H = 128$ hash functions the standard deviation of the estimate is approximately 0.044.

```python
class MinHasher:
    PRIME = 2_147_483_647

    def __init__(self, n_hash: int, seed: int = 42):
        lcg = LCG(seed=seed)
        pairs = lcg.generate_pairs(n_hash, lo=1, hi=self.PRIME)
        self.a_coeffs = [p[0] for p in pairs]
        self.b_coeffs = [p[1] for p in pairs]

    def signature(self, shingle_set: set) -> list:
        if not shingle_set:
            return [0] * self.n_hash
        shingles_list = list(shingle_set)
        sig = []
        for i in range(self.n_hash):
            a, b, p = self.a_coeffs[i], self.b_coeffs[i], self.PRIME
            min_val = p
```

```
            for x in shingles_list:
                hval = (a * x + b) % p
                if hval < min_val:
                    min_val = hval
            sig.append(min_val)
        return sig

    def estimated_jaccard(self, sig_a: list, sig_b: list) -> float:
        matches = sum(1 for va, vb in zip(sig_a, sig_b) if va == vb)
        return matches / self.n_hash
```

Listing 3: MinHash signature and Jaccard estimate (Cell 8)

5.4. Step 3 — Locality Sensitive Hashing (Cell 9, MMDS §3.4)

The `LSH` class splits each 128-element signature into `N_BANDS = 32` bands of `ROWS_PER_BAND = 4` rows (auto-computed as `N_HASH // N_BANDS`). In `fit()`, for each band, the 4 signature values of every document are stored as a tuple key in a Python dictionary. In `candidate_pairs()`, any two documents sharing the same key in at least one band are added to the candidate set as a sorted `(i, j)` tuple.

```
class LSH:
    def fit(self, signatures: list):
        self.buckets = [dict() for _ in range(self.b)]
        for band_idx in range(self.b):
            row_start = band_idx * self.r
            row_end   = row_start + self.r
            for doc_idx in range(len(signatures)):
                key = (band_idx,) + tuple(
                    signatures[doc_idx][row_start:row_end])
                if key not in self.buckets[band_idx]:
                    self.buckets[band_idx][key] = []
                self.buckets[band_idx][key].append(doc_idx)

    def candidate_pairs(self) -> set:
        candidates = set()
        for band_dict in self.buckets:
            for bucket in band_dict.values():
                if len(bucket) > 1:
                    sd = sorted(bucket)
                    for ii in range(len(sd)):
                        for jj in range(ii+1, len(sd)):
                            candidates.add((sd[ii], sd[jj]))
        return candidates
```

Listing 4: LSH banding and candidate extraction (Cell 9)

After building the index, the notebook prints the total possible pairs, the number of candidates, and the percentage reduction. The theoretical LSH threshold is $(1/b)^{1/r}$; with $b = 32$ and $r = 4$, `theoretical_thresh` ≈ 0.42.

5.5. Step 4 — Candidate Verification (Cell 10)

`find_similar_pairs` iterates over all candidate pairs, calls `minhash.estimated_jaccard` on their signatures, and retains only pairs with estimated Jaccard $\geq$ `SIMILARITY_THRESHOLD = 0.5`. Results are sorted by similarity descending. The notebook prints the number of candidates processed, time taken, and the final count of similar pairs found.

6. Scalability

6.1. Theoretical Complexity

Step	Complexity	Notes
Shingling	$O(n \cdot L)$	L = average comment length
MinHashing	$O(n \cdot H \cdot \bar{S})$	$\bar{S}$ = average shingle set size
LSH indexing	$O(n \cdot H)$	Linear in n
Candidate extraction	$O(n \cdot b)$	Iterating over band dictionaries
Candidate verification	$O(\lvert C\rvert \cdot H)$	$\lvert C\rvert \ll n^2$ in practice
Total (LSH)	$O(n \cdot H \cdot \bar{S})$	Near-linear in n
Brute-force	$O(n^2 \cdot H)$	Quadratic — infeasible at scale

6.2. Design Decisions for Scale

1. **Selective loading:** only the four needed columns are read via `usecols`; row count is capped via `nrows`. Raising `MAX_COMMENTS` scales up without changing any algorithm code.
2. **Integer shingle sets:** hash integers are stored rather than raw strings, reducing memory substantially.
3. **Parallel lists:** signatures and metadata are stored as flat Python lists giving $O(1)$ index access.
4. **LSH dictionary-of-lists:** each band uses a `dict` with $O(1)$ average insert and lookup; total memory $O(n \cdot b)$.
5. **No all-pairs step:** no step compares all $\binom{n}{2}$ pairs; within-bucket comparisons are negligible in size.

6.3. Empirical Runtime

Each major step records elapsed time with `time.time()`. At `MAX_COMMENTS = 30,000` on Google Colab free CPU:

Step	Approx. time
Load and clean 30K comments	15–25 s
Shingling ($k = 5$)	30–50 s
MinHashing ($H = 128$)	40–60 s
LSH indexing ($b = 32$, $r = 4$)	< 2 s
Candidate verification	< 1 s
Total	$\approx$ 90–130 s

The dominant cost is MinHashing, whose inner per-shingle loop runs in pure Python. A vectorised NumPy approach would reduce it 50–100× without changing the algorithm.

7. Experiments

7.1. Configuration

Parameter (variable)	Value
Dataset	NYT Comments 2020, accessed March 2025
Corpus size (`N_DOCS`)	up to 30,000 comments
Minimum comment length	80 characters
Shingle size (`K_SHINGLE`)	5
Signature length (`N_HASH`)	128
LSH bands (`N_BANDS`)	32
Rows per band (`ROWS_PER_BAND`)	4 (auto-computed)
LSH threshold (`theoretical_thresh`)	≈ 0.42
Reporting threshold (`SIMILARITY_THRESHOLD`)	0.5
Top pairs displayed (`TOP_N`)	15
Random seed (`RANDOM_SEED`)	42
Platform	Google Colab (CPU)

7.2. Candidate Reduction

For a corpus of 30,000 comments the total possible pairs are $\binom{30000}{2} = 449{,}985{,}000$. LSH reduces this to a very small number of candidates (well under 0.001% of all pairs). The exact counts are printed by Cell 9 as a percentage reduction, confirming that the overwhelming majority of dissimilar comment pairs are never compared.

7.3. Similar Pairs Found and Displayed (Cell 11)

`display_pair` prints the top `TOP_N = 15` pairs. For each pair it outputs:

- Rank, MinHash estimated Jaccard (4 decimal places), and true Jaccard computed by `jaccard_similarity` on the actual shingle sets;
- A 20-character visual bar representing the estimated similarity;
- Comment ID and article ID for each of the two comments;
- The first 250 characters of each comment body, with `...` appended if truncated;
- Count of shared shingles, total shingles, and whether both comments belong to the same article.

Qualitatively, found pairs fall into three categories:

1. **Exact duplicates** (estimated Jaccard near 1.0): the same comment posted verbatim under different articles.
2. **Near-duplicates** (estimated Jaccard 0.7–0.95): comments sharing the same template, differing in a few words.
3. **Topical matches** (estimated Jaccard 0.5–0.7): different users writing about the same story in similar language.

7.4. Similarity Distribution (Cell 12)

Cell 12 produces a histogram of MinHash Jaccard similarity scores across all found pairs, with a red dashed vertical line at `SIMILARITY_THRESHOLD = 0.5`. The chart is titled with the total number of pairs found. Note that `plt.show()` is commented out: the figure is generated into a

`matplotlib Axes` object but is not explicitly rendered. It will display automatically in Colab's inline backend when the cell is executed.

7.5. Results Summary (Cell 13)

Cell 13 prints a final summary covering: corpus size, shingle size, signature length, LSH configuration, theoretical threshold, reporting threshold, total pairs, candidate count, percentage reduction, and similar pairs found.

8. Discussion

8.1. Correctness

The MinHash theorem guarantees the estimated Jaccard is unbiased. With $H = 128$ the standard deviation is approximately 0.044. For every displayed pair, `display_pair` computes the exact Jaccard from the shingle sets alongside the estimate, providing a direct per-pair verification that the MinHash values are accurate.

8.2. Limitations

1. **Pure Python performance.** The MinHash inner loop is the runtime bottleneck. NumPy vectorisation would reduce it 50–100× without changing the algorithm.
2. **Short comments.** Comments near the 80-character threshold produce small shingle sets (∼50–80 elements), making estimates noisier.
3. **No quantitative evaluation.** The notebook contains no brute-force ground-truth comparison. Internal correctness is confirmed only for the top-15 displayed pairs via the true Jaccard printed by `display_pair`; overall precision and recall are not measured.
4. **Threshold gap.** Pairs with true Jaccard in [0.42, 0.5) may become LSH candidates but are filtered out by `SIMILARITY_THRESHOLD = 0.5`. Lowering the threshold would discover more pairs at the cost of more false positives.
5. `user_display` **computed but unused.** Cell 4 derives `user_display` from `userDisplayName`, and Cell 5 includes it in `corpus_df`, but it is never extracted into a parallel list and the corresponding output lines in Cell 11 are commented out.

8.3. Possible Extensions

- Add a brute-force evaluation cell on a small subset to measure precision and recall quantitatively;
- Either remove `user_display` from Cell 4 and Cell 5 entirely, or uncomment the user print lines in Cell 11 to actually use it;
- Experiment with word-level shingles ($k = 2$ or $k = 3$ words) for broader semantic similarity;
- Cross-reference found pairs with `article_refs` to distinguish same-article verbatim reposts from cross-article recurring viewpoints.

9. Conclusion

This project implements the complete Shingling–MinHash–LSH pipeline from MMDS Chapter 3 on the NYT Comments 2020 dataset across 14 code cells with no external algorithm libraries. The pipeline reduces 449 million possible comment pairs to a small candidate set and identifies pairs with estimated Jaccard similarity of 0.5 or above. For each found pair the notebook displays full comment text, article references, and the exact Jaccard alongside the MinHash estimate, directly confirming correctness. The corpus size is governed by a single global variable, no step performs an all-pairs comparison, and the notebook runs end-to-end in approximately two minutes on a free Colab CPU instance.

A. Global Parameters Reference

Variable	Default	Description
`SUBSAMPLE`	`True`	Enables subsampling to `MAX_COMMENTS`
`MAX_COMMENTS`	30,000	Total comments to load and process
`K_SHINGLE`	5	Character shingle window size
`N_HASH`	128	MinHash signature length
`N_BANDS`	32	LSH bands; must divide `N_HASH` evenly
`ROWS_PER_BAND`	4	Auto-computed: `N_HASH // N_BANDS`
`SIMILARITY_THRESHOLD`	0.5	Minimum estimated Jaccard to report a pair
`TOP_N`	15	Number of top similar pairs to display
`RANDOM_SEED`	42	Seed for reproducible shuffling

B. Declaration

"I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study."